

Балтийский государственный технический университет
«ВОЕНМЕХ» им. Д. Ф. Устинова

Кафедра О7 «Информационные системы и программная инженерия»

Практическая работа №2
по дисциплине «Структуры и организация данных»
на тему «Нелинейные структуры данных»

вариант 12

Выполнил:
Студент Праудин Д. С.
Группа О717

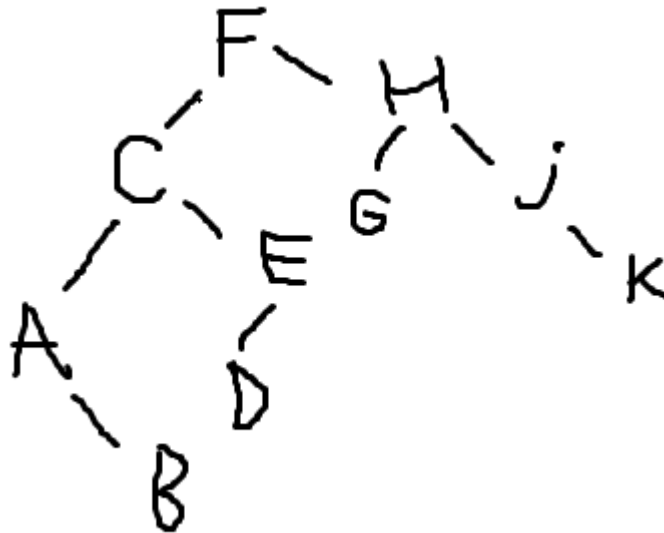
Преподаватель:
Палехова О. А.

Санкт-Петербург
2022 г.

Задание 1.

В прямом порядке обхода бинарного дерева была получена последовательность узлов FCABEDHGJK, а в симметричном - ABCDEFGHJK. Какие узлы этого дерева имеют двух сыновей?

Полученное дерево:



Ответ: C, H, F

Задание 2.

Вопрос 1

Не завершено

Балл: 3,00

Отметить вопрос

Оставшееся время 0:59:44

1

[Закончить попытку...](#)

В некоторой библиотеке есть два класса, в которых определены операции для работы с неориентированным взвешенным графом.

В первом классе граф представлен матрицей весов, структура хранения которой – треугольная динамическая матрица, схематичное изображение которой приведено на рисунке. Элементы матрицы могут принимать целые неотрицательные значения, указатель `m` объявлен так:

```
unsigned int ** m;
```

Во втором классе граф представляется списками смежности, структура хранения каждого из которых – односвязный линейный список, значением поля данных каждого элемента является номер вершины и вес дуги. Указатели на головы списков размещаются в массиве. Схематичное изображение структуры хранения в целом приведено на рисунке ниже. Указатель `g` объявлен так:

```
struct node  
{  
    short int number_node;  
    unsigned int weight;  
    node* next;  
} **g;
```

```

{
    short int number_node;
    unsigned int weight;
    node* next;
} **g;

```

Укажите минимальный порядок графа, для хранения которого в объекте второго класса будет выделяться гарантированно меньше памяти, чем в объекте первого класса, если степень любой вершины не будет превышать семи, sizeof (int) и размер каждого указателя - 4 байта, а поля структуры располагаются вплотную друг к другу.

Ответ:

Решение задачи:

Рассмотрим размер 1 класса:

$$S_1 = S_{11} + S_{12} + S_{13}$$

Где S_1 – размер 1 класса в байтах, S_{11} – размер указателя на массив указателей, S_{12} – размер массива указателей на массивы, S_{13} – размер массивов с весами.

$S_{11} = 4$ байта – размер указателя на массив указателей

$S_{12} = 4 * N = 4N$ байта - размер массива указателей, где N – количество вершин графа

$S_{13} = 4 * (N^2 - N) / 2 = 2N^2 - 2N$ байта – размер массивов с весами графа

$$S_1 = 4 + 4N + 2N^2 - 2N = 2N^2 + 2N + 4 \text{ байта}$$

Рассмотрим размер 2 класса:

$$S_2 = S_{21} + S_{22} + S_{23}$$

Где S_2 – размер 2 класса в байтах, S_{21} – размер указателя на массив указателей на списки, S_{22} – размер массива указателей на списки, S_{23} – размер списков.

$S_{21} = 4$ байта – размер указателя на массив указателей на списки

$S_{22} = 4 * N = 4N$ байта - размер массива указателей на списки, где N – количество вершин графа

$S_{23} = (2 + 4 + 4) * 7 * N = 70N$ байта – размер списков

$$S_2 = 4 + 4N + 70N = 74N + 4 \text{ байта – размер 2 класса}$$

$$S_2 < S_1$$

$$74N + 4 < 2N^2 + 2N + 4$$

$$N^2 - 36N > 0 \text{ (} N \text{ всегда больше или равно 0, потому что это количество вершин)}$$

$$N > 36 \Rightarrow N_{\min} = 37$$

Ответ: 37

Задание 3. Написать программу для решения задачи.

Уровень сложности – **повышенный**. Граф представляется двумя способами (матрицей смежности или весов и списками смежности), для тестирования программы требуется

создать файлы с описанием графа одним способом (только матрицей или только списками), при считывании данных из файла заполнять параллельно обе структуры хранения (и реализующую представление матрицы смежности (весов), и реализующую представление списков смежных вершин). При выборе структур хранения руководствоваться требованием разумной экономии памяти.

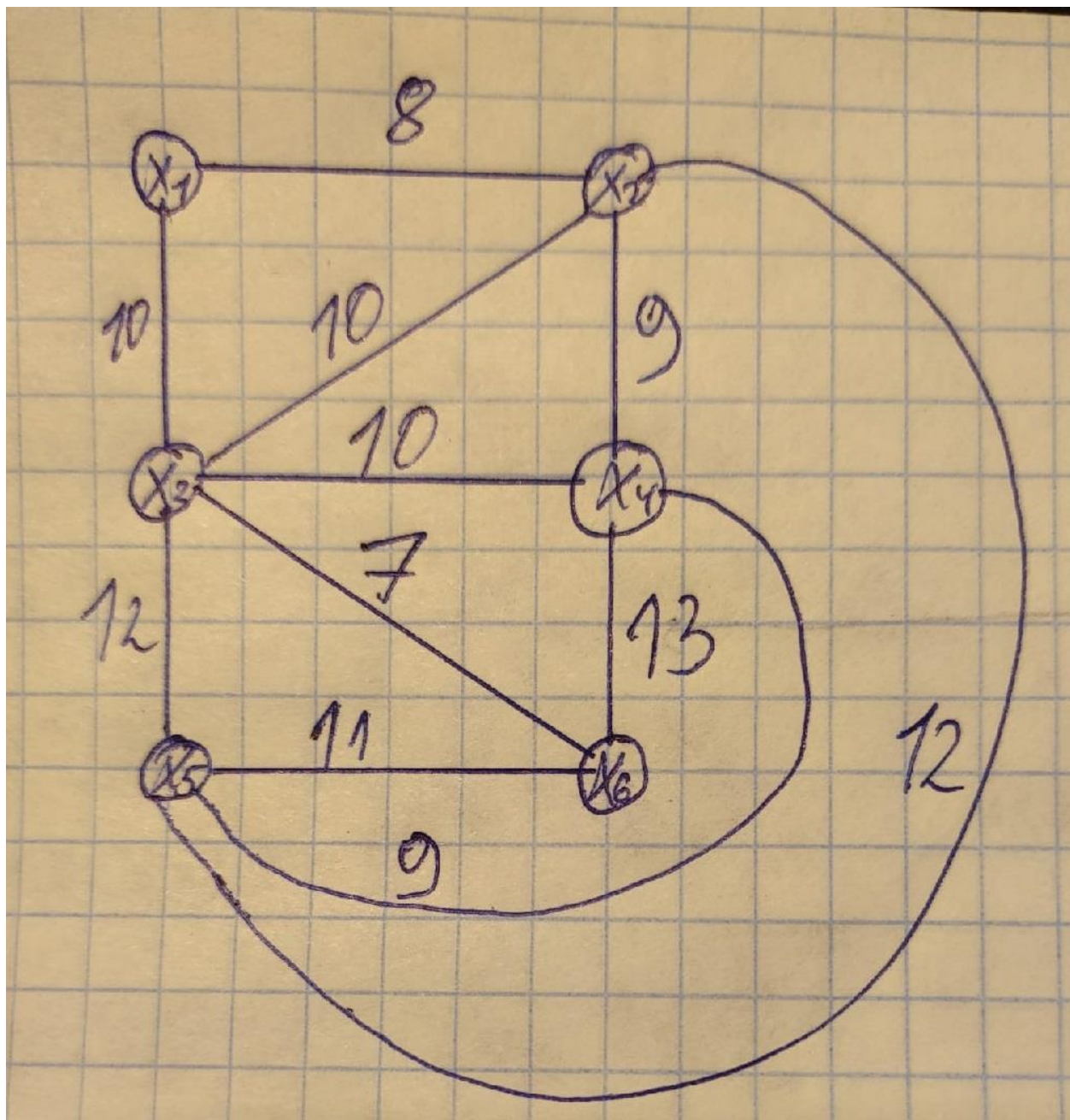
В системе двусторонних дорог за проезд каждой дороги взимается некоторая пошлина.

Найти путь из города А в город В с минимальной величиной $S + P$, где S – сумма длин дорог пути, а P – сумма пошлин проезжаемых дорог. А и В вводятся с клавиатуры. На среднем и повышенном уровнях сложности помимо найденного пути вывести отдельно S и P .

Тестовый граф:

На рисунках нарисованы графы с указанием сумм $S + P$, конкретные значения S и P указаны в файлах .

Граф 1:



GraphL.txt (представление списком смежности):

"6 1

2 2 4 4 3 6 4

4 1 4 4 3 2 8 4 5 4 5 6 6

5 1 6 4 2 2 8 4 7 3 5 7 5 6 4 3

4 2 5 4 3 7 3 5 3 6 6 7 6

4 2 6 6 3 7 5 4 3 6 6 9 2

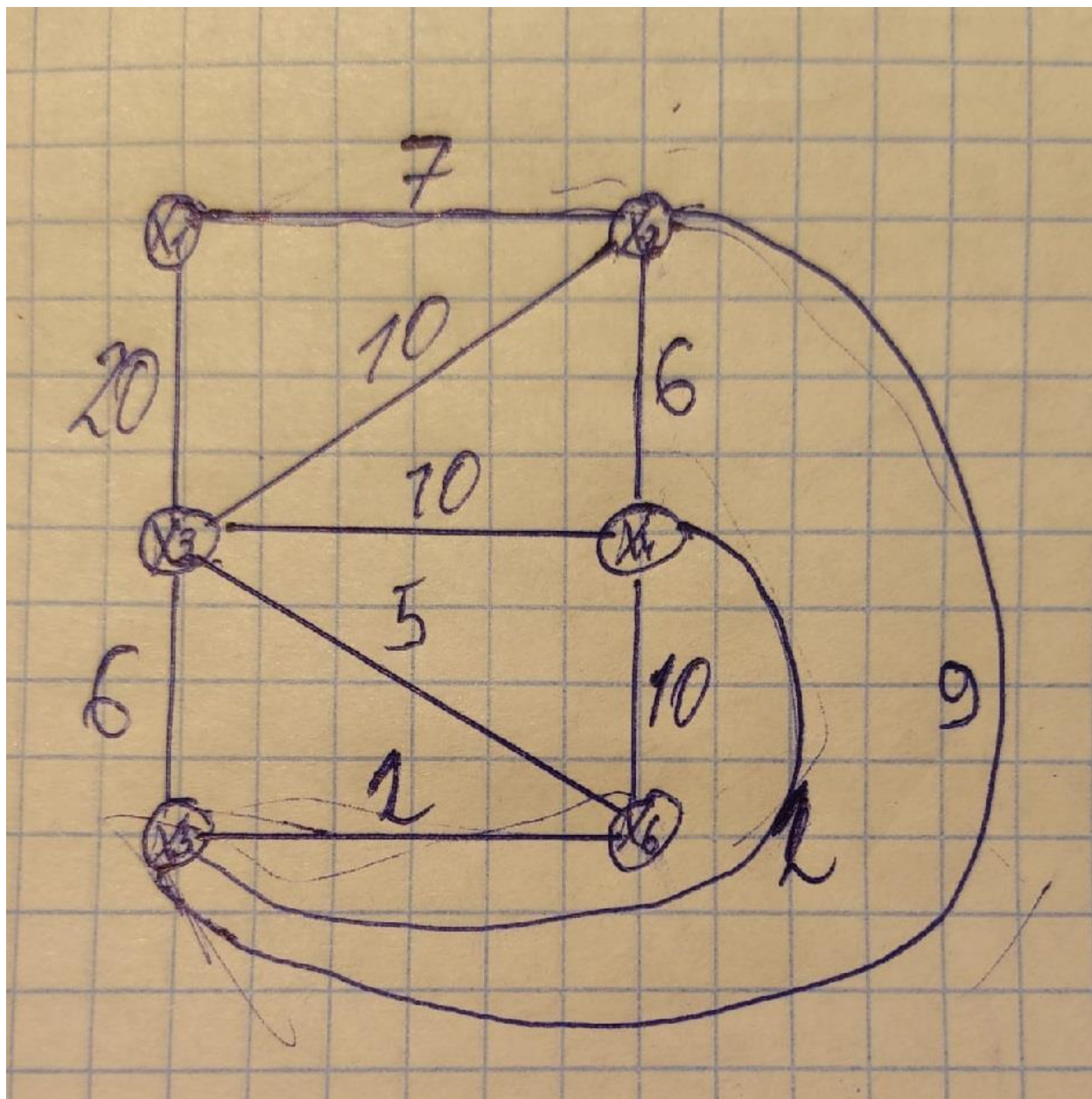
3 3 4 3 4 7 6 5 9 2"

GraphV.txt (представление матрицей весов):

"6 2

004464000000
 440028546600
 642800737543
 005473003676
 006675360092
 000043769200"

Граф 2:



GraphL.txt (представление списком смежности):

"6 1

2 2 3 4 3 1 1 9

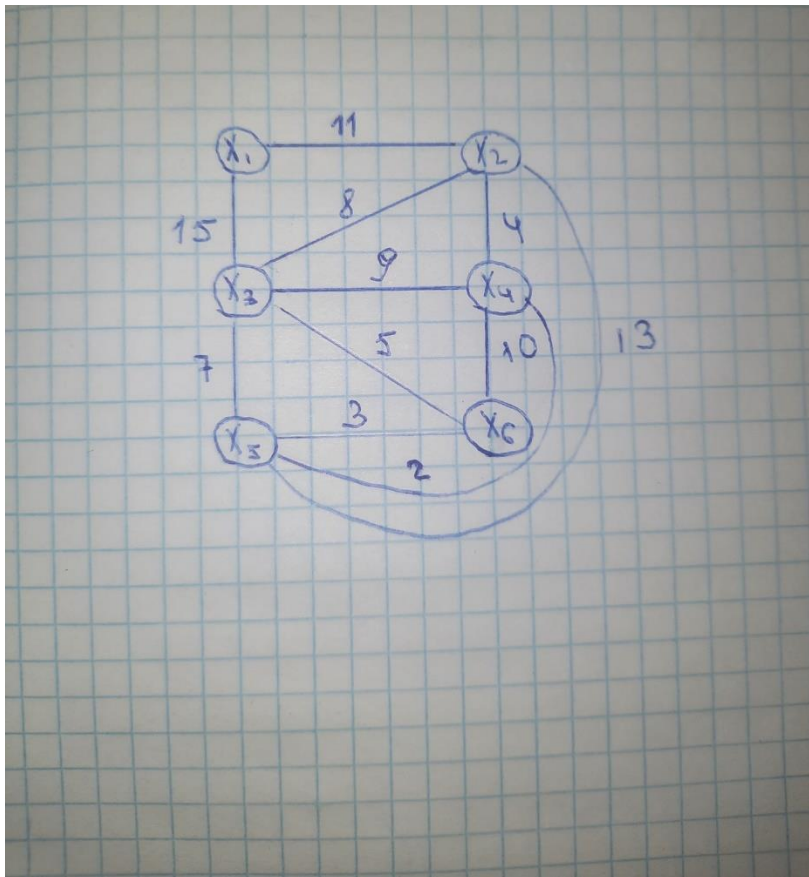
4 1 3 4 3 5 5 4 5 1 5 4 5

5 1 11 9 2 5 5 4 7 3 5 2 4 6 2 3
 4 2 5 1 3 7 3 5 1 1 6 7 3
 4 2 4 5 3 7 3 4 1 1 6 1 1
 3 3 2 3 4 7 3 5 1 1”

GraphV.txt (представление матрицей весов):

“6 2
 0 0 3 4 1 1 9 0 0 0 0 0 0
 3 4 0 0 5 5 5 1 4 5 0 0
 1 1 9 5 5 0 0 7 3 2 4 2 3
 0 0 5 1 7 3 0 0 1 1 7 3
 0 0 4 5 7 3 1 1 0 0 1 1
 0 0 0 0 2 3 7 3 1 1 0 0”

Граф 3:



GraphL.txt (представление списком смежности):

“6 1
 2 2 6 5 3 9 6
 4 1 6 5 3 4 4 4 2 2 5 8 5
 5 1 9 6 2 4 4 4 5 4 5 4 3 6 3 2

4 2 2 2 3 5 4 5 1 1 6 5 5

4 2 8 5 3 4 3 4 1 1 6 2 1

3 3 3 2 4 5 5 5 2 1”

GraphV.txt (представление матрицей весов):

“6 2

0 0 6 5 9 6 0 0 0 0 0 0

6 5 0 0 4 4 2 2 8 5 0 0

9 6 4 4 0 0 5 4 4 3 3 2

0 0 2 2 5 4 0 0 1 1 5 5

0 0 8 5 4 3 1 1 0 0 2 1”

0 0 0 0 3 2 5 5 2 1 0 0

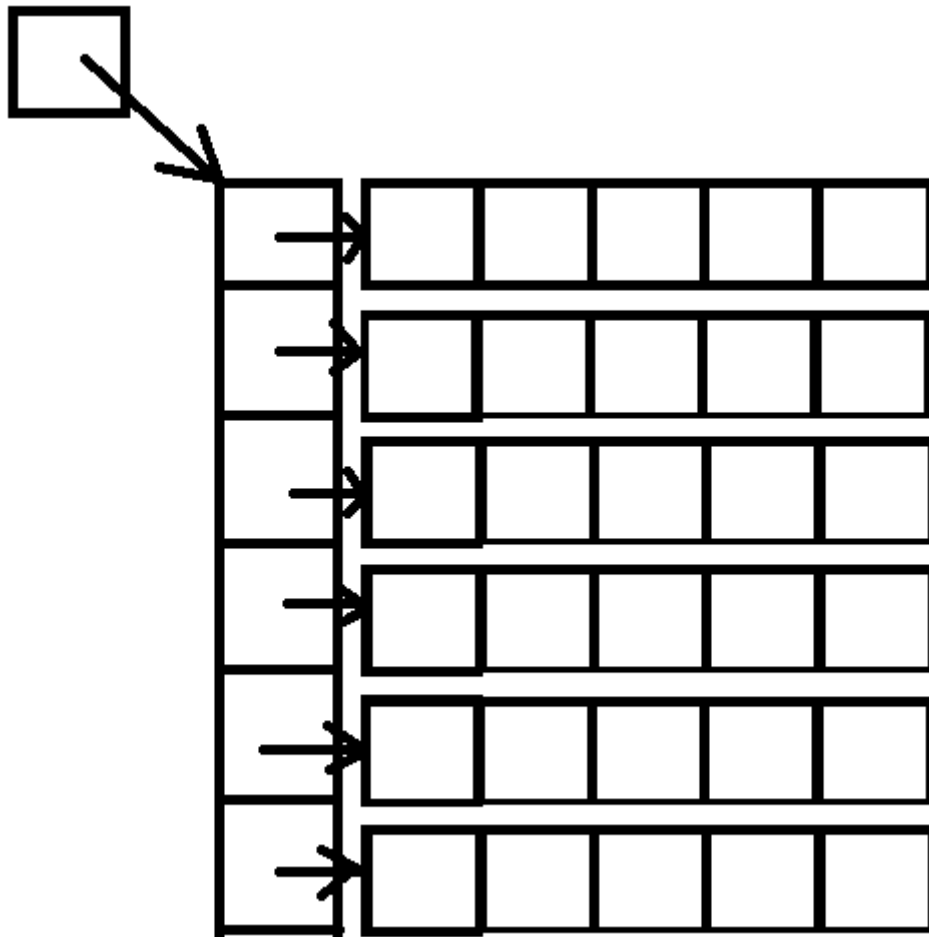
Ожидаемый результат:

Для Графа 1: $d(x_1, x_6) = 17$, $S = 10$, $P = 7$.

Для Графа 2: $d(x_1, x_6) = 17$, $S = 10$, $P = 7$.

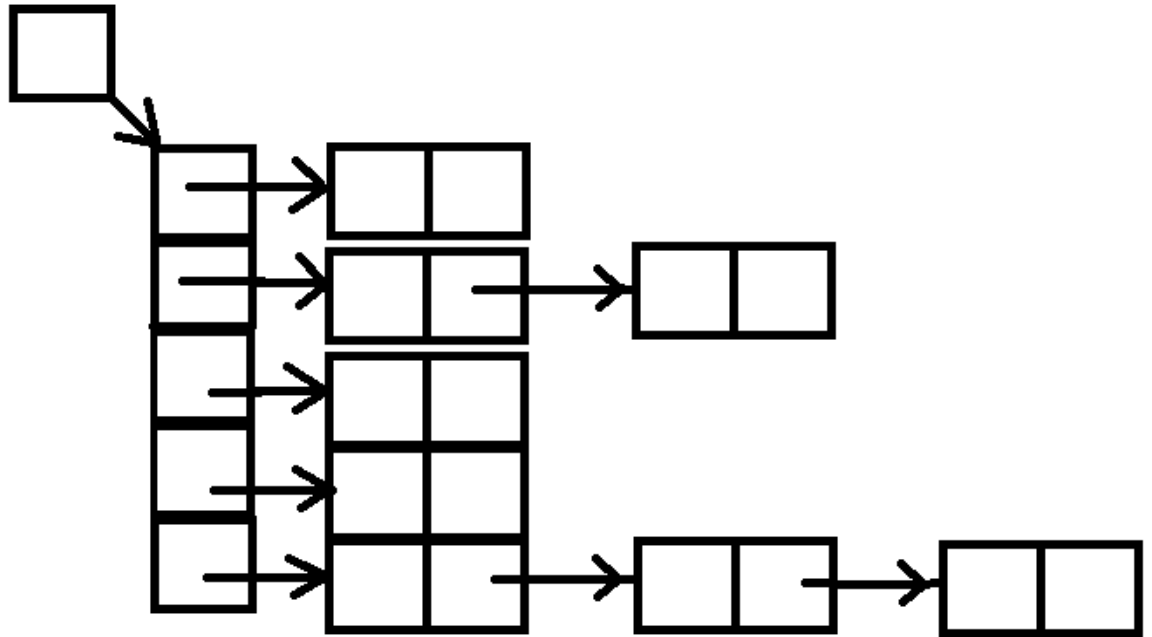
Для Графа 3: $d(x_1, x_6) = 20$, $S = 12$, $P = 8$.

Структура хранения для представления графа матрицей смежности или весов:



Данная структура хранения удобна при небольших количествах вершин графов, так как при них занимает меньший объём в памяти и обеспечивает быстрое взаимодействие с данными.

Структура хранения для представления графа списками смежных вершин:



Данная структура удобна при больших количествах вершин графа, потому что будет занимать значительно меньше места в памяти, чем матричное представление. Однако скорость доступа к данным вершины будет меньше, чем в матрице.

Применяемые алгоритмы:

Для представления графа списком смежных вершин был использован алгоритм Дейкстры, а для матричного представления графа был использован алгоритм Флойда-Уоршелла.

Текст программы:

С представлением графа списком смежных вершин:

Z2_1.cpp:

```
#include <iostream>
#include <Windows.h>
#include "List.h"

int main(int argc, char* argv[]) {
    SetConsoleCP(1251);
    SetConsoleOutputCP(1251);
    ListG graf;
    char filename[80];
    if (argc > 1)
        strcpy(filename, argv[1]);
    else {
        std::cout << "Введите имя файла: ";
        std::cin >> filename;
    }
    if (graf.ReadFile(filename))
        std::cout << "Граф считан." << std::endl;
    else {
        std::cout << "Ошибка при считывании графа." << std::endl;
        return -1;
    }
}
```

```

        system("pause");
        system("cls");
        std::cout << "Данные графа: " << std::endl;
        graf.PrintG();
        int na, co;
        std::cout << "Введите номер стартовой вершину: ";
        std::cin >> na;
        std::cout << "Введите номер конечной вершину: ";
        std::cin >> co;
        graf.FindMin(na, co);
        return 0;
    }
}
List.h:
#pragma once

struct node {
    int v, s, p, w;
    void Fill(int v, int s, int p); //заполнение узла
    node* next;
};

struct el {
    int s, p, w;
    el(); //конструктор структуры...
    el(const el& e); //конструктор копирования
    void SetEl(int s, int p, int w); //задание значений структуры
    el& operator=(const el& el); //перезагрузка =
    friend el operator+(el& el1, el& el2); //перезагрузка +
};

class ListG {
private:
    int kol;
    node** aop;
public:
    ListG(); //конструктор по умолчанию
    ~ListG(); //деструктор
    bool ReadFile(const char* s); //считывание данных о графе в список
    смежности
    void PrintG(); //метод печати графа
    void FindMin(int na, int fn); //метод нахождения пути между двумя
    вершинами
    void AddNode(int i, node* t, node* n); //добавление узла
    node* FindP(int wm, int v); //нахождение расстояния из текущей вершины
};
List.cpp:
#include <iostream>
#include "List.h"

el::el() {
    s = 99999;
    p = 99999;
    w = 99999;
}

el::el(const el& e) {
    this->p = e.p;
    this->s = e.s;
    this->w = e.w;
}

el& el::operator=(const el& el) {
    this->p = el.p;

```

```

        this->s = el.s;
        this->w = el.w;
        return *this;
    }

void el::SetEl(int s, int p, int w) {
    this->s = s;
    this->p = p;
    this->w = w;
}

el operator+(el& el1, el& el2) {
    el l;
    l.s = el1.s + el2.s;
    l.p = el1.p + el2.p;
    l.w = el1.w + el2.w;
    return l;
}

void node::Fill(int v, int s, int p) {
    this->next = nullptr;
    this->v = v;
    this->s = s;
    this->p = p;
    this->w = s + p;
}

ListG::ListG() {
    this->kol = 0;
    this->aop = nullptr;
}

ListG::~~ListG() {
    for (int i = 0; i < this->kol; i++) {
        node* tmp;
        while (this->aop[i] != nullptr) {
            tmp = this->aop[i];
            this->aop[i] = this->aop[i]->next;
            delete tmp;
        }
    }
    delete[] this->aop;
}

bool ListG::ReadFile(const char* s) {
    FILE* f;
    f = fopen(s, "r");
    if (f == nullptr) {
        std::cout << "Ошибка открытия файла!!!" << std::endl;
        return false;
    }
    int tmp, tmp1, tp, ts, tv, mt;
    node* temp = nullptr;
    tmp1 = fscanf(f, "%d %d\n", &kol, &mt); // считываем первую строку из
    файла
    if (tmp1 < 2 || kol <= 0)
        return false;
    this->aop = new node * [this->kol]; // выделим память под массив
    указателей
    if (this->aop == nullptr) {
        std::cout << "Ошибка при выделении памяти." << std::endl;
        fclose(f);
        return false;
    }
}

```

```

    }
    for (int i = 0; i < this->kol; i++) //
        this->aop[i] = nullptr;
    if (mt == 1) {
        for (int i = 0; i < this->kol; i++) {
            tmp1 = fscanf(f, "%d", &tmp);
            if (tmp1 < 1 || tmp <= 0) {
                fclose(f);
                return false;
            }
            for (int j = 0; j < tmp; j++) {
                tmp1 = fscanf(f, " %d %d %d", &tv, &ts, &tp);
                if (tmp1 < 3 || tv <= 0 || ts <= 0 || tp <= 0) {
                    fclose(f);
                    return false;
                }
                node* n = new node;
                if (n == nullptr) {
                    std::cout << "Ошибка при выделении памяти." <<
std::endl;

                    fclose(f);
                    return false;
                }
                n->Fill(tv, ts, tp);
                AddNode(i, temp, n);
                temp = n;
            }
        }
    }
    else if (mt == 2) {
        for (int i = 0; i < this->kol; i++) {
            for (int j = 0; j < this->kol; j++) {
                tmp1 = fscanf(f, "%d %d", &ts, &tp);
                if (tmp1 < 2 || ts < 0 || tp < 0) {
                    fclose(f);
                    return false;
                }
                if (ts > 0 && tp > 0) {
                    node* n = new node;
                    if (n == nullptr) {
                        std::cout << "Ошибка при выделении
памяти." << std::endl;

                        fclose(f);
                        return false;
                    }
                    n->Fill(j + 1, ts, tp);
                    AddNode(i, temp, n);
                    temp = n;
                }
            }
        }
    }
    else {
        fclose(f);
        return false;
    }
    fclose(f);
    return true;
}

void ListG::PrintG() {
    for (int i = 0; i < this->kol; i++) {
        std::cout << i + 1 << " :\t";
    }
}

```

```

        node* temp = this->aop[i];
        while (temp != nullptr) {
            std::cout << temp->v << " " << temp->w << '\t';
            temp = temp->next;
        }
        std::cout << '\n';
    }
}

void ListG::FindMin(int na, int fn) {
    int* s, wm, v;
    el* d;
    node* tmp, * c;
    s = new int[this->kol];
    d = new el[this->kol];
    if (s == nullptr || d == nullptr) {
        if (s != nullptr)
            delete[] s;
        if (d != nullptr)
            delete[] d;
        std::cout << "Ошибка при выделении памяти." << std::endl;
        return;
    }
    for (int i = 0; i < this->kol; i++) {
        s[i] = 0;
    }
    s[na - 1] = 1; // массив с 0 а вершины с 1
    tmp = this->aop[na - 1]; // записываем адрес первого элемента
    списка соответствующего введённой вершине
    while (tmp != nullptr) { // пробегаемся по всему списку и с помощью
    функции SetEl переписываем данные из каждого узла списка в соответствующий
    элемент массива структур el
        d[tmp->v - 1].SetEl(tmp->s, tmp->p, tmp->w);
        tmp = tmp->next;
    }
    for (int i = 2; i < this->kol; i++) {
        for (wm = na - 1, v = 0; v < this->kol; v++)
            if (!s[v] && d[v].w < d[wm].w) // сравниваем сумму первого
            узла и начального (ищем минимальную)
                wm = v;
        if (wm == fn - 1)
            break;
        s[wm] = 1; // вершина рассмотрена
        for (v = 0; v < this->kol; v++) {
            c = FindP(wm, v);
            if (v != na - 1 && c != nullptr && d[wm].w + c->w < d[v].w)
            { // сумма минимальной и текущей рассматриваемой
                el tr;
                tr.SetEl(c->s, c->p, c->w); // заполняем узел найденным
                d[v] = d[wm] + tr; // складываем две структуры
            }
        }
    }
    if (d[fn - 1].w != 99999)
        std::cout << "min(S + P) = " << d[fn - 1].w << "\nS = " << d[fn -
1].s << "\nP = " << d[fn - 1].p << std::endl;
    else
        std::cout << "Нет минимального пути." << std::endl;
}

node* ListG::FindP(int wm, int v) { // нахождение нужной вершины в нужном
списке
    node* tmp = this->aop[wm]; // адрес начала нужного списка

```

```

        while (tmp != nullptr && tmp->v - 1 != v) //пока список не кончился
(строка)
            tmp = tmp->next;
        return tmp;
    }

void ListG::AddNode(int i, node* t, node* n) {
    if (this->aop[i] == nullptr) // если список для текущей строки файла
пуст, то в него записывается n - новый созданный узел, а если не пуст, то n
прикрепляется к предыдущему узлу
        this->aop[i] = n;
    else
        t->next = n;
}

```

Для представления графа матрицей весов:

Z2_2.cpp:

```

#include <iostream>
#include <Windows.h>
#include "Vec.h"

using namespace std;

int main(int argc, char* argv[]) {
    SetConsoleCP(1251);
    SetConsoleOutputCP(1251);
    VecG graf;
    char filename[80];
    if (argc > 1)
        strcpy(filename, argv[1]);
    else {
        cout << "Введите имя файла: ";
        cin >> filename;
    }
    if (graf.ReadFile(filename))
        cout << "Граф считан." << std::endl;
    else {
        cout << "Ошибка при считывании графа." << std::endl;
        return -1;
    }
    system("pause");
    system("cls");
    std::cout << "Данные графа: " << std::endl;
    graf.PrintG();
    int na, co;
    cout << "Введите номер стартовой вершину: ";
    cin >> na;
    cout << "Введите номер конечной вершину: ";
    cin >> co;
    graf.FindMin(na, co);
    return 0;
}

GVec.h:
#pragma once

struct node {
    int s, p, w;
    node(); //конструктор по умолчанию для структуры
    node(const node& n); //конструктор копирования
    void Fill(int s1, int p1); //заполнение узла
    friend node operator+(node& n1, node& n2); //перегрузка +
    node& operator=(const node& n); //перегрузка =
};

```



```

class VecG {
private:
    int kol;
    node** aop;
public:
    VecG();//конструктор по умолчанию
    ~VecG();//деструктор
    bool ReadFile(const char* s);//считывание файла
    void PrintG();//метод печати графа
    void FindMin(int na, int fn);//метод поиска минимального пути
};

Vec.cpp:
#include <iostream>
#include "Vec.h"

node::node() {
    this->s = 99999;
    this->p = 99999;
    this->w = 99999;
}

node::node(const node& n) {
    this->s = n.s;
    this->p = n.p;
    this->w = n.w;
}

node operator+(node& n1, node& n2) {
    node n;
    n.s = n1.s + n2.s;
    n.p = n1.p + n2.p;
    n.w = n1.w + n2.w;
    return n;
}

node& node::operator=(const node& n) {
    this->s = n.s;
    this->p = n.p;
    this->w = n.w;
    return *this;
}

void node::Fill(int s1, int p1) {
    this->s = s1;
    this->p = p1;
    this->w = s1 + p1;
}

VecG::VecG() {
    this->kol = 0;
    this->aop = nullptr;
}

VecG::~VecG() {
    if (this->aop != nullptr) {
        for (int i = 0; i < this->kol; i++)
            if (this->aop[i] != nullptr)
                delete[] this->aop[i];
        delete[] this->aop;
    }
}

bool VecG::ReadFile(const char* s) {

```

```

FILE* f;
f = fopen(s, "r");
if (f == nullptr) {
    std::cout << "Ошибка открытия файла!!!" << std::endl;
    return false;
}
int tmp1, mt, tmp, tv, ts, tp;
tmp1 = fscanf(f, "%d %d\n", &this->kol, &mt);
if (tmp1 < 2 || this->kol <= 0) {
    fclose(f);
    return false;
}
this->aop = new node * [this->kol]; // выделим память под массив
указателей
if (this->aop == nullptr) {
    std::cout << "Ошибка при выделении памяти." << std::endl;
    fclose(f);
    return false;
}
for (int i = 0; i < this->kol; i++) {
    this->aop[i] = new node[this->kol]; // под каждый в отдельности
выделяем память
    if (this->aop[i] == nullptr) { // если память под какой то массив
не выделилась удаляем все выделенное и удаляем массивы указателей
        for (int j = i; j >= 0; j--) // проходимся по строке в
обратном порядке удаляя
            delete[] this->aop[j];
        delete[] this->aop;
        fclose(f);
        return false;
    }
}
if (mt == 1) {
    for (int i = 0; i < this->kol; i++) {
        tmp1 = fscanf(f, "%d", &tmp);
        if (tmp1 < 1 || tmp <= 0) {
            fclose(f);
            return false;
        }
        for (int j = 0; j < tmp; j++) {
            tmp1 = fscanf(f, " %d %d %d", &tv, &ts, &tp);
            if (tmp1 < 3 || tv <= 0 || ts <= 0 || tp <= 0) {
                fclose(f);
                return false;
            }
            this->aop[i][tv - 1].Fill(ts, tp); // записываем в
узел
        }
    }
}
else if (mt == 2) {
    for (int i = 0; i < this->kol; i++) {
        for (int j = 0; j < this->kol; j++) {
            tmp1 = fscanf(f, "%d %d", &ts, &tp);
            if (tmp1 < 2 || ts < 0 || tp < 0) {
                fclose(f);
                return false;
            }
            if (ts > 0 && tp > 0) {
                this->aop[i][j].Fill(ts, tp);
            }
        }
    }
}

```

```

    }
    else {
        fclose(f);
        return false;
    }
    fclose(f);
    return true;
}

void VecG::FindMin(int na, int fn) {
    for (int k = 0; k < this->kol; k++)
        for (int i = 0; i < this->kol; i++)
            for (int j = 0; j < this->kol; j++)
                if (this->aop[i][j].w > this->aop[i][k].w + this->aop[k][j].w)
                    this->aop[i][j] = this->aop[i][k] + this->aop[k][j];
    if (this->aop[na - 1][fn - 1].w != 99999)
        std::cout << "min(S + P) = " << this->aop[na - 1][fn - 1].w <<
        "\nS = " << this->aop[na - 1][fn - 1].s << "\nP = " << this->aop[na - 1][fn - 1].p << std::endl;
    else
        std::cout << "Нет минимального пути." << std::endl;
}

void VecG::PrintG() {
    for (int i = 0; i < this->kol; i++) {
        for (int j = 0; j < this->kol; j++)
            std::cout << this->aop[i][j].w << '\t';
        std::cout << '\n';
    }
}

```

Результаты работы программы:

1 программа для Графа 1:

```

Данные графа:
1 :    2 8    3 10
2 :    1 8    3 10    4 9    5 12
3 :    1 10    2 10    4 10    5 12    6 7
4 :    2 9    3 10    5 9    6 13
5 :    2 12    3 12    4 9    6 11
6 :    3 7    4 13    5 11
Введите номер стартовой вершину: 1
Введите номер конечной вершину: 6
min(S + P) = 17
S = 10
P = 7

```

2 программа для Графа 1:

```

Данные графа:
99999 8      10      99999 99999 99999
8      99999 10      9      12      99999
10     10      99999 10      12      7
99999 9      10      99999 9      13
99999 12     12      9      99999 11
99999 99999 7      13      11      99999
Введите номер стартовой вершину: 1
Введите номер конечной вершину: 6
min(S + P) = 17
S = 10
P = 7

```

1 программа для Графа 2:

```

Данные графа:
1 :      2 7      3 20
2 :      1 7      3 10      4 6      5 9
3 :      1 20     2 10      4 10     5 6      6 5
4 :      2 6      3 10      5 2      6 10
5 :      2 9      3 10      4 2      6 2
6 :      3 5      4 10      5 2
Введите номер стартовой вершину: 1
Введите номер конечной вершину: 6
min(S + P) = 17
S = 10
P = 7

```

2 программа для Графа 2:

```

Данные графа:
99999 7      20      99999 99999 99999
7      99999 10      6      9      99999
20     10      99999 10      6      5
99999 6      10      99999 2      10
99999 9      10      2      99999 2
99999 99999 5      10      2      99999
Введите номер стартовой вершину: 1
Введите номер конечной вершину: 6
min(S + P) = 17
S = 10
P = 7

```

1 программа для Графа 3:

```

Данные графа:
1 :      2 11      3 15
2 :      1 11      3 8      4 4      5 13
3 :      1 15      2 8      4 9      5 7      6 5
4 :      2 4      3 9      5 2      6 10
5 :      2 13      3 7      4 2      6 3
6 :      3 5      4 10      5 3
Введите номер стартовой вершину: 1
Введите номер конечной вершину: 6
min(S + P) = 20
S = 12
P = 8

```

2 программа для Графа 3:

```
Данные графа:
99999  11      15      99999  99999  99999
11      99999  8        4        13      99999
15      8      99999  9        7        5
99999  4        9      99999  2        10
99999  13      7        2        99999  3
99999  99999  5        10      3        99999
Введите номер стартовой вершину: 1
Введите номер конечной вершину: 6
min(S + P) = 20
S = 12
P = 8
```